

Ταξινόμηση σαφήνειας απαντήσεων σε πολιτικά QA ζεύγη

Student name: *Αλέξανδρος Γκιάφης*
AM: *sdi2200284*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025–2026*

Περιεχόμενα

1	Περίληψη	2
2	Επεξεργασία και ανάλυση δεδομένων	2
2.1	Προεπεξεργασία	2
2.2	Ανάλυση δεδομένων	3
2.3	Διαχωρισμός σε train, validation και test	4
2.4	Διανυσματοποίηση / αναπαραστάσεις	5
3	Αλγόριθμοι και πειράματα	5
3.1	Πειραματικό setup	5
3.2	Αντιπροσωπευτικά experiments	6
3.3	Hyperparameter tuning	6
3.4	Πρακτικές επιλογές και optimization	7
3.5	Αξιολόγηση	7
3.5.1	Learning curves	8
3.5.2	Confusion matrix και error analysis	9
4	Αποτελέσματα και συνολική ανάλυση	10
4.1	Καλύτερο validation trial και τελικό submit	10
4.2	Συνολική ερμηνεία των αποτελεσμάτων	11
4.3	Περιορισμοί και πιθανές επεκτάσεις	12
5	Συμπεράσματα	12
6	Βιβλιογραφία	13

1. Περίληψη

Στην εργασία αυτή ασχολήθηκα με το task της ταξινόμησης της σαφήνειας μιας απάντησης σε πολιτικά ζεύγη ερώτησης-απάντησης από το SemEval 2026 Task 6 (CLARITY). Για κάθε QA pair το μοντέλο πρέπει να προβλέψει μία από τις τρεις κλάσεις: *Clear Reply*, *Ambivalent* ή *Clear Non-Reply*. Όπως ζητούσε και η εκφώνηση, υλοποίησα δύο βασικές οικογένειες αναπαράστασης: **TF-IDF + Logistic Regression** και **Word2Vec + Logistic Regression**. Πέρα από τα βασικά text representations, δοκίμασα και διάφορα engineered features που προσπαθούν να πιάσουν καλύτερα τη σχέση ανάμεσα στην ερώτηση και την απάντηση, όπως overlap, question intent, answer evidence και QA alignment.

Από τα experiments φάνηκε αρκετά καθαρά ότι τα TF-IDF μοντέλα δούλεψαν καλύτερα από τα Word2Vec μοντέλα στο validation. Η καλύτερη συνολικά οικογένεια ήταν η **qa_dual_tfidf**, δηλαδή μια αναπαράσταση όπου question και answer μοντελοποιούνται ξεχωριστά και μετά συνδυάζονται με πρόσθετα QA-aware features. Το καλύτερο validation αποτέλεσμα που είδα ήταν accuracy 0.6521 και macro-F1 0.6058. Για το τελικό submit χρησιμοποίησα το configuration **text_plus_all_features + qa_dual_tfidf** με **en_core_web_sm**, το οποίο έδωσε **public score 0.66** και με έφερε στην **top-3** του leaderboard.

Το βασικό συμπέρασμα είναι ότι σε αυτό το task δεν αρκεί να έχεις απλώς μια “καλή” αναπαράσταση του κειμένου. Πιο πολύ μετράει αν το μοντέλο καταλαβαίνει τη σχέση ερώτησης-απάντησης: αν δηλαδή η απάντηση όντως απαντάει, αν απαντάει μόνο μερικώς ή αν αποφεύγει εντελώς το ζητούμενο. Εκεί τα sparse TF-IDF features, ειδικά μαζί με QA-aware engineered features, αποδείχθηκαν πιο χρήσιμα από τα απλά averaged Word2Vec embeddings.

2. Επεξεργασία και ανάλυση δεδομένων

2.1. Προεπεξεργασία

Το αρχικό training split είχε 3448 γραμμές. Μετά τον καθαρισμό έμειναν 3434 έγκυρα παραδείγματα, αφού αφαιρέθηκαν 14 corrupted rows που είχαν εμφανή θόρυβο και μη ρεαλιστικά OOV patterns. Το test set είχε 308 παραδείγματα.

Στο preprocessing χρησιμοποίησα το profile **old_plus_number_token**. Πρακτικά αυτό σήμαινε:

1. lowercasing και βασικό cleanup του whitespace,
2. normalization επιφανειακών patterns, π.χ. ημερομηνίες, ώρες, ποσά, percentages, numeric ranges και standalone numbers,
3. lemmatization με spaCy [4],
4. προστασία ορισμένων ειδικών tokens ώστε να μη χαλάνε στο lemmatization,
5. δημιουργία ξεχωριστών processed στηλών για question, answer και combined QA text.

Για το spaCy δοκίμασα τόσο το **en_core_web_sm** όσο και το **en_core_web_trf**. Τελικά κράτησα το **en_core_web_sm**, γιατί στην πράξη το **trf** είχε πολύ μεγαλύτερο runtime χωρίς να δίνει ουσιαστικό improvement στα scores. Αυτό φάνηκε και στα tests που έκανα: το **sm** ήταν αρκετά πιο “τίμιο” σαν επιλογή για full runs και submit.

Παράλληλα με το text preprocessing, έφτιαξα και αρκετά engineered features. Οι βασικές ομάδες ήταν οι εξής:

- **surface / punctuation features:** αν υπάρχουν αριθμοί, ποσοστά, money, χρονικές αναφορές, question marks κ.λπ.,
- **question intent features:** αν η ερώτηση μοιάζει με *what*, *why*, *when*, *who*, *yes/no* κτλ.,
- **answer evidence features:** αν η απάντηση περιέχει κάποια πιο συγκεκριμένη πληροφορία, π.χ. χρόνο, ποσότητα ή αιτιολόγηση,
- **QA alignment features:** αν ο τύπος της απάντησης ταιριάζει με το τι ζητάει η ερώτηση,
- **overlap / pair-structure features:** lexical overlap, bigram overlap, length ratios και γενικά ενδείξεις για το πόσο “κουμπώνει” η απάντηση με την ερώτηση,
- **answer-group features:** πληροφορία για περιπτώσεις όπου η ίδια απάντηση επαναχρησιμοποιείται σε πολλές σχετικές ερωτήσεις.

Η λογική πίσω από όλο αυτό το preprocessing δεν ήταν μόνο να “καθαρίσω” το κείμενο. Ήθελα κυρίως να κρατήσω cues που είναι χρήσιμα για το συγκεκριμένο task. Σε πολιτικά interviews, πολλές φορές η αποφυγή απάντησης ή η μισή απάντηση δεν φαίνεται μόνο από το θέμα της πρότασης, αλλά από πιο τοπικά και δομικά patterns.

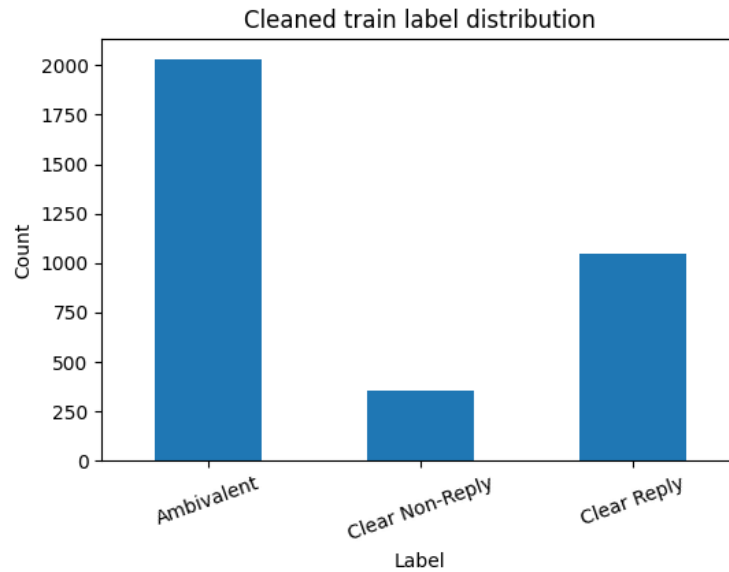
2.2. Ανάλυση δεδομένων

Από το EDA φάνηκε εξ αρχής ότι το dataset είναι **ανισόρροπο**. Στο καθαρισμένο training set, η κλάση *Ambivalent* είναι το 59.1% των παραδειγμάτων, η *Clear Reply* το 30.5% και η *Clear Non-Reply* μόλις το 10.3%. Αυτός είναι και ο βασικός λόγος που δεν αρκεί να κοιτάμε μόνο accuracy. Για model selection έδωσα μεγαλύτερη σημασία στο **macro-F1**.

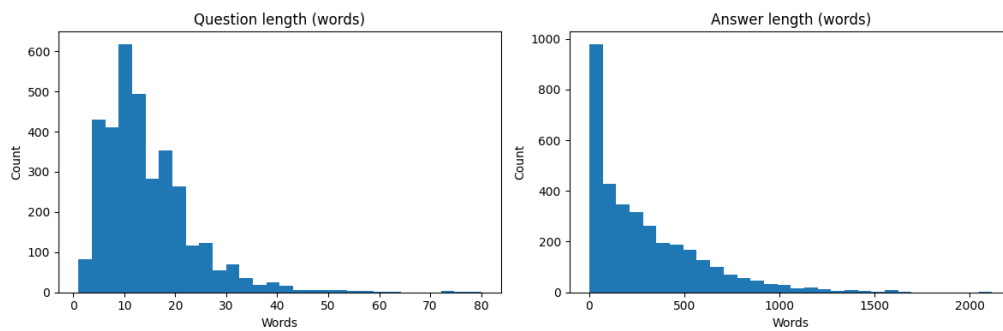
Ένα δεύτερο ενδιαφέρον σημείο ήταν η μεγάλη διαφορά στο μήκος question και answer. Η μέση ερώτηση είχε περίπου 14.45 tokens, ενώ η μέση απάντηση περίπου 290.45. Άρα αν απλώς ενώσω question και answer σε ένα string, η απάντηση κυριαρχεί σχεδόν πάντα. Αυτό ήταν και βασικό motivation για τα dual representations, όπου question και answer περνάνε από ξεχωριστές προβολές.

Στους τύπους ερωτήσεων, οι πιο συχνές κατηγορίες ήταν *yes/no* (1389 παραδείγματα) και *other* (1132), ενώ μετά έρχονταν τα *what* και *how*. Αυτό ενίσχυσε την ιδέα ότι τα question-intent features έχουν νόημα, γιατί μια καλή απάντηση σε *when* ερώτηση δεν μοιάζει με μια καλή απάντηση σε *why* ή *yes/no* ερώτηση.

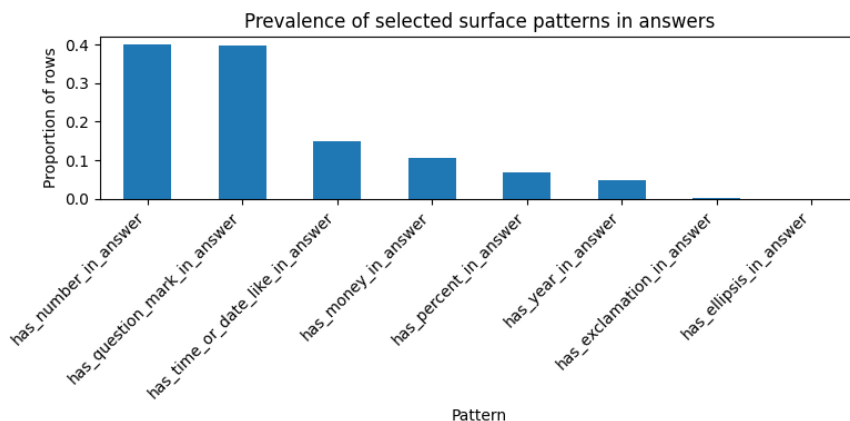
Στις απαντήσεις εμφανίζονταν αρκετά συχνά surface patterns όπως αριθμοί (39.9%), question marks (39.6%), time/date-like patterns (15.0%), money (10.5%) και percentages (6.8%). Επίσης, αυτά τα patterns δεν ήταν μοιρασμένα ισότιμα στις κλάσεις. Για παράδειγμα, η *Ambivalent* είχε πιο συχνά αριθμητικά και χρονικά στοιχεία από την *Clear Non-Reply*. Αυτό δείχνει ότι η *Ambivalent* δεν είναι “καθαρή αποφυγή”, αλλά αρκετές φορές μοιάζει με απάντηση που λέει κάτι σχετικό χωρίς να απαντάει τελείως καθαρά.



Σχήμα 1: Κατανομή labels στο καθαρισμένο training set. Η κλάση *Ambivalent* είναι με διαφορά η μεγαλύτερη.



Σχήμα 2: Κατανομή μήκους ερωτήσεων και απαντήσεων. Οι απαντήσεις είναι πολύ μεγαλύτερες από τις ερωτήσεις.



Σχήμα 3: Συχνότητα επιλεγμένων surface patterns στις απαντήσεις.

2.3. Διαχωρισμός σε train, validation και test

Για όλα τα experiments χρησιμοποίησα **stratified split 80/20** με **random_state=42**. Έτσι, από τα 3434 training examples προέκυψαν 2747 για subtrain και 687 για validation. Το stratification κράτησε σχεδόν ίδιες τις αναλογίες των τριών classes και στα δύο splits, οπότε οι συγκρίσεις ανάμεσα στα runs ήταν πιο δίκαιες.

Προτίμησα να δουλέψω με σταθερό validation split σε όλα τα βασικά experiments, γιατί έτσι οι διαφορές που βλέπω οφείλονται κυρίως στα features και στα model settings και όχι σε αλλαγές του split. Αργότερα, για το καλύτερο candidate, έκανα και 5-fold cross-validation σαν extra check σταθερότητας.

2.4. Διανυσματοποίηση / αναπαραστάσεις

Υλοποιήθηκαν τέσσερις βασικές εκδοχές αναπαράστασης:

1. **tfidf**: baseline concatenation του question και του answer σε ένα κοινό sparse TF-IDF space.
2. **qa_dual_tfidf**: ξεχωριστά TF-IDF blocks για question και answer, μαζί με shared QA space, answer openings, character n-grams και similarity features.
3. **word2vec**: dense αναπαράσταση 200 διαστάσεων με average token embeddings, με βάση τη γνωστή ιδέα των word vectors [1].
4. **qa_dual_word2vec**: ξεχωριστά averaged embeddings για question και answer, μαζί με similarity και προαιρετικά engineered features.

Στην πράξη, το dual TF-IDF setup ήταν αυτό που δούλεψε καλύτερα. Ο λόγος μάλλον είναι ότι κρατάει πολύ καθαρά τα τοπικά lexical patterns και τα n-grams που ξεχωρίζουν μια σαφή απάντηση από μια αμφίσημη ή evasive απάντηση. Αντίθετα, το Word2Vec έχει το πλεονέκτημα της dense σημασιολογικής αναπαράστασης, αλλά εδώ φάνηκε να χάνει αρκετή από τη λεπτομέρεια που χρειάζεται το task.

3. Αλγόριθμοι και πειράματα

3.1. Πειραματικό setup

Σε όλα τα βασικά experiments ο classifier ήταν **Logistic Regression** από **scikit-learn** [2]. Αυτό ήταν και σύμφωνο με την εκφώνηση, αλλά και πρακτικά λογικό: πρόκειται για έναν δυνατό και γρήγορο linear classifier που δουλεύει καλά τόσο σε sparse όσο και σε dense text features.

Για το hyperparameter tuning έκανα grid search στις τιμές:

$$C \in \{0.05, 0.1, 0.2, 0.5, 1, 2, 5, 10\}, \quad \text{class_weight} \in \{\text{None}, \text{balanced}\},$$

με solver **liblinear** και **max_iter=3000**. Το βασικό criterion για επιλογή μοντέλου ήταν το **validation macro-F1**.

Δεν περιορίστηκα μόνο σε ένα feature setup. Δοκίμασα αρκετά experiment presets, όπως **text_only**, **text_plus_all_features**, **text_plus_structure_only**, **text_plus_surface** και **text_plus_relational_core**. Στόχος ήταν να δω όχι μόνο ποιο representation family είναι καλύτερο, αλλά και ποια groups από engineered features δίνουν το πιο χρήσιμο signal.

3.2. Αντιπροσωπευτικά experiments

Ο Πίνακας 1 δείχνει μερικά από τα πιο αντιπροσωπευτικά runs.

Experiment	Representation	Accuracy	Macro-F1	Σχόλιο
text_only	tfidf	0.6157	0.5557	baseline sparse model
text_only	qa_dual_tfidf	0.6405	0.5832	καθαρό κέρδος από το dual QA setup
text_only	word2vec	0.5837	0.4778	αδύναμο dense baseline
text_only	qa_dual_word2vec	0.6084	0.5143	βελτίωση, αλλά ακόμα κάτω από TF-IDF
text_plus_all_features	tfidf	0.6245	0.5847	μικρό gain από engineered features
text_plus_all_features	qa_dual_tfidf	0.6405	0.5933	πολύ καλή και σταθερή submit-ready λύση
text_plus_all_features	word2vec	0.6157	0.5296	βελτιωμένο αλλά όχι αρκετό
text_plus_all_features	qa_dual_word2vec	0.6390	0.5138	καλή accuracy, αρκετά πιο αδύναμο macro-F1
text_plus_relational_core	qa_dual_tfidf	0.6492	0.6017	καλύτερο stage-1 validation model
Stage-2 refined	qa_dual_tfidf	0.6521	0.6058	καλύτερο συνολικό validation result

Πίνακας 1: Αντιπροσωπευτικά experiments από τις βασικές οικογένειες αναπαραστάσεων και feature presets.

Το πιο σημαντικό που φαίνεται από τον πίνακα είναι ότι το πέρασμα από **tfidf** σε **qa_dual_tfidf** δίνει σταθερό improvement σχεδόν παντού. Άρα δεν βοηθάει απλώς το να έχουμε καλύτερα features, αλλά ειδικά το ότι question και answer μοντελοποιούνται ξεχωριστά. Από την άλλη, τα Word2Vec models βελτιώνονται όταν μπαίνουν σε dual setup και όταν προστίθενται engineered features, αλλά πάλι δεν φτάνουν τα καλύτερα TF-IDF runs.

3.3. Hyperparameter tuning

Το hyperparameter tuning έγινε κυρίως πάνω στη Logistic Regression. Στην πράξη, οι επιλογές που έβγαιναν πιο συχνά καλές ήταν:

- solver **liblinear**, που αποδείχθηκε σταθερός σε sparse, high-dimensional spaces,
- μεσαίες προς σχετικά μεγάλες τιμές του C , ανάλογα με το feature space,

- συχνή χρήση του `class_weight=balanced`, ειδικά επειδή η *Clear Non-Reply* είναι minority class.

Μετά το πρώτο round experiments έκανα και ένα **stage-2 refinement** στους καλύτερους candidates. Εκεί έγιναν πιο στοχευμένες αλλαγές στο dual TF-IDF setup, όπως πιο αυστηρό `min_df` σε κάποια blocks, tuning στα answer character n-grams και μικρές βελτιώσεις σε answer-opening features. Αυτό ανέβασε το καλύτερο validation result από **macro-F1 0.6017** σε **0.6058** και το accuracy από **0.6492** σε **0.6521**. Δεν είναι τεράστια διαφορά, αλλά ήταν σταθερό improvement και επιβεβαίωσε ότι το καλύτερο ceiling βρισκόταν όντως στην οικογένεια `qa_dual_tfidf`.

3.4. Πρακτικές επιλογές και optimization

Πέρα από το tuning, υπήρχαν και μερικές πρακτικές επιλογές που βοήθησαν αρκετά:

1. **Caching** ενδιάμεσων **artifacts**, ώστε να μπορώ να ξανατρέχω experiments χωρίς να ξαναχτίζω κάθε φορά όλο το pipeline.
2. Σταθερό **random seed** σε preprocessing, split και modeling, ώστε τα runs να είναι αναπαραγωγίμα.
3. Χρήση του **en_core_web_sm** αντί για **en_core_web_trf**, επειδή το **trf** ήταν πολύ πιο αργό χωρίς πραγματικό gain.
4. Έμφαση στα **QA-relational features** αντί να αυξάνω απλώς ανεξέλεγκτα το feature space.

Το τελευταίο σημείο είναι σημαντικό: το καλύτερο μοντέλο δεν ήταν απαραίτητα αυτό με τα περισσότερα δυνατά features, αλλά αυτό που είχε τα πιο σχετικά features για τη σχέση question-answer.

3.5. Αξιολόγηση

Η βασική μετρική επιλογής ήταν το **macro-F1**, επειδή το πρόβλημα είναι multiclass και ανισόρροπο. Εκτός από αυτό, κατέγραψα και **accuracy**, **macro precision**, **macro recall**, καθώς και **weighted metrics**.

Η accuracy από μόνη της μπορεί να είναι παραπλανητική. Φάνηκε σε αρκετά Word2Vec runs, όπου το accuracy ήταν σχετικά καλό, αλλά το macro-F1 έμενε αρκετά χαμηλά. Αυτό σημαίνει ότι το μοντέλο μπορεί να τα πήγαινε καλά στην πλειοψηφική κλάση *Ambivalent*, αλλά όχι τόσο καλά στις υπόλοιπες, ειδικά στη *Clear Non-Reply*.

Στον Πίνακα 2 φαίνεται το καλύτερο validation result ανά βασική αναπαράσταση.

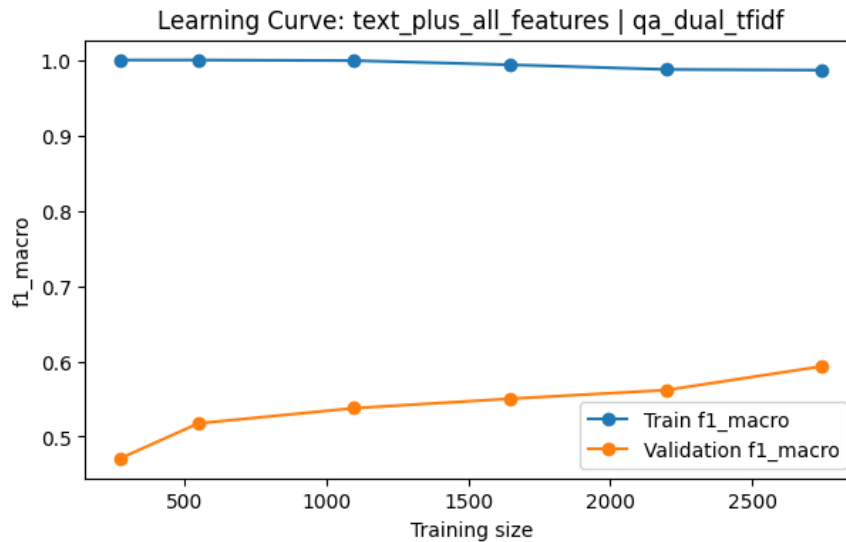
Representation	Καλύτερο experiment	Accuracy	Macro-F1	Παρατήρηση
qa_dual_tfidf	text_plus_relational_core	0.6492	0.6017	καλύτερη οικογένεια συνολικά
tfidf	text_plus_structure_only	0.6259	0.5873	ισχυρό sparse baseline
word2vec	text_plus_all_features	0.6157	0.5296	βελτιώνεται με features αλλά μένει πιο πίσω
qa_dual_word2vec	text_plus_relational_core	0.6041	0.5251	το dual setup μόνο του δεν κλείνει το gap

Πίνακας 2: Καλύτερο validation result ανά βασική αναπαράσταση.

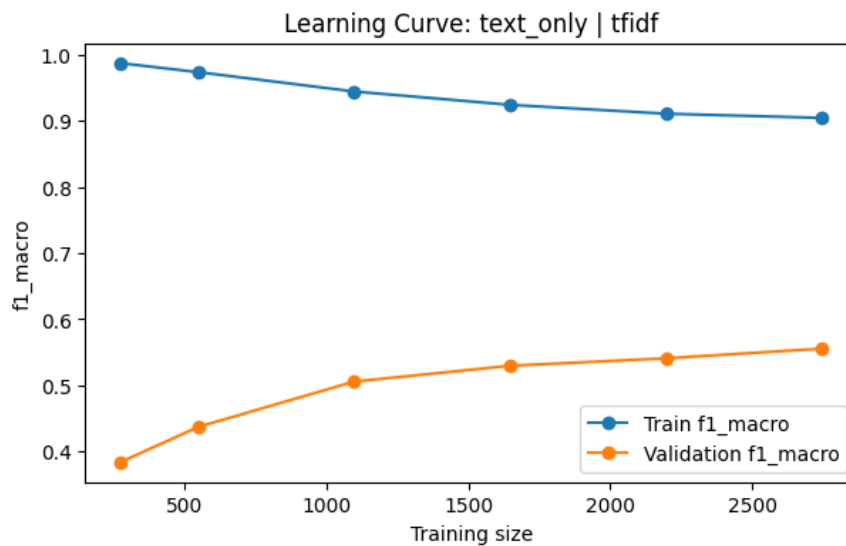
Από αυτόν τον πίνακα φαίνεται καθαρά ότι το **qa_dual_tfidf** είναι η καλύτερη επιλογή overall. Δεν είναι μόνο το καλύτερο TF-IDF variant, αλλά ξεπερνάει και όλα τα Word2Vec-based variants με αισθητή διαφορά στο macro-F1. Αυτό ταιριάζει και με τη γενική εικόνα του task: το μοντέλο χρειάζεται να πιάσει πολύ λεπτά lexical και structural cues, και εκεί τα sparse n-grams μάλλον έχουν πλεονέκτημα.

3.5.1. Learning curves. Οι learning curves έδειξαν δύο βασικά πράγματα. Πρώτον, τα δυνατά sparse models πιάζουν πολύ υψηλό training score ακόμα και με μικρό training size, άρα έχουν αρκετή χωρητικότητα. Δεύτερον, το validation macro-F1 συνεχίζει να ανεβαίνει όσο μεγαλώνει το training set και δεν φαίνεται να έχει κορεστεί τελείως. Άρα υπάρχει χώρος για βελτίωση, είτε με περισσότερο data είτε με πιο στοχευμένο regularization.

Στο **text_plus_all_features + qa_dual_tfidf** το validation macro-F1 ανεβαίνει περίπου από 0.47 σε train size 274 σε περίπου 0.59 στο πλήρες subtrain size 2747. Αντίστοιχα, το απλό **text_only + tfidf** ξεκινά χαμηλότερα και καταλήγει περίπου στο 0.56. Αυτό δείχνει ότι το dual setup μαζί με τα engineered features αξιοποιεί καλύτερα και την αύξηση των δεδομένων.



Σχήμα 4: Learning curve για το `text_plus_all_features + qa_dual_tfidf`.



Σχήμα 5: Learning curve για το baseline `text_only + tfidf`.

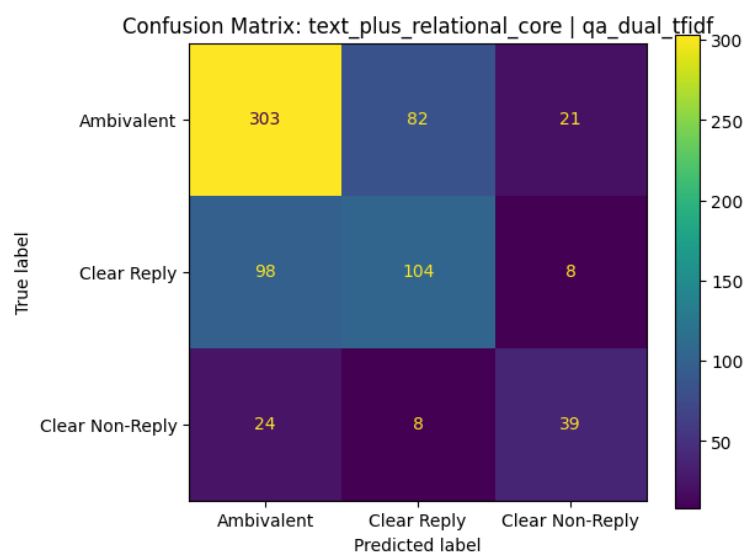
3.5.2. Confusion matrix και error analysis. Για πιο αναλυτικό error analysis κοίταξα το καλύτερο stage-1 validation model, δηλαδή το `text_plus_relational_core + qa_dual_tfidf`. Το μοντέλο αυτό είχε accuracy 0.6492 και macro-F1 0.6017, με την εξής per-class εικόνα:

- **Ambivalent:** precision 0.7129, recall 0.7463, F1 0.7292
- **Clear Reply:** precision 0.5361, recall 0.4952, F1 0.5149
- **Clear Non-Reply:** precision 0.5735, recall 0.5493, F1 0.5612

Η πιο εύκολη κλάση ήταν η *Ambivalent*. Αυτό είναι αναμενόμενο, και επειδή είναι η μεγαλύτερη class, αλλά και επειδή πολλές φορές έχει επαναλαμβανόμενα patterns. Η πιο δύσκολη ήταν η *Clear Reply*, κυρίως επειδή μπερδεύονταν αρκετά με την *Ambivalent*. Οι βασικές συγχύσεις ήταν:

- *Clear Reply* → *Ambivalent*: 98 περιπτώσεις
- *Ambivalent* → *Clear Reply*: 82 περιπτώσεις
- *Clear Non-Reply* → *Ambivalent*: 24 περιπτώσεις
- *Ambivalent* → *Clear Non-Reply*: 21 περιπτώσεις

Αυτό το pattern είναι αρκετά λογικό. Οι περισσότερες αστοχίες δεν γίνονται ανάμεσα στα δύο “άκρα”, δηλαδή *Clear Reply* και *Clear Non-Reply*, αλλά γύρω από την ενδιάμεση κλάση *Ambivalent*. Με άλλα λόγια, το δύσκολο κομμάτι του task δεν είναι να ξεχωρίσεις μια καθαρή απάντηση από μια εντελώς άσχετη αποφυγή. Το δύσκολο είναι να ξεχωρίσεις πότε μια απάντηση λέει κάτι σχετικό, αλλά τελικά δεν απαντάει αρκετά καθαρά.



Σχήμα 6: Confusion matrix για το καλύτερο stage-1 validation model. Οι μεγαλύτερες συγχύσεις είναι μεταξύ *Ambivalent* και *Clear Reply*.

4. Αποτελέσματα και συνολική ανάλυση

4.1. Καλύτερο validation trial και τελικό submit

Το καλύτερο validation result προέκυψε από ένα refined qa_dual_tfidf model με preset text_plus_relational_core. Οι βασικές μετρικές του ήταν:

- accuracy = 0.6521
- macro precision = 0.6103
- macro recall = 0.6017
- macro-F1 = 0.6058

Σε 5-fold cross-validation, ο ίδιος candidate είχε μέσο macro-F1 περίπου 0.5785 με standard deviation 0.0145. Άρα η λύση ήταν καλή και σχετικά σταθερή, αλλά το task παραμένει δύσκολο και έχει κάποια διακύμανση ανά split.

Παρόλα αυτά, το **τελικό model που χρησιμοποιήσα για submit** δεν ήταν ακριβώς αυτό το validation-best candidate. Για submit χρησιμοποίησα την παραλλαγή:

text_plus_all_features + qa_dual_tfidf (dual_base)

με Logistic Regression ρυθμίσεις `C=2.0, solver=liblinear, max_iter=4000, class_weight=balanced, penalty=l2`, και spaCy model `en_core_web_sm`. Παρότι αυτό δεν ήταν το απόλυτο best validation setup, ανήκε στην πιο δυνατή οικογένεια μοντέλων, ήταν πολύ σταθερό στα runs και τελικά έδωσε **public Kaggle score 0.66**, που αντιστοιχούσε σε 3η θέση στο leaderboard.

Η υποβολή έγινε από τον Αλέξανδρο Γκιάφη (AM `sdi2200284`), με Kaggle competition nickname **AlexTuring** - `sdi2200284`. Το notebook της λύσης μπορεί να προσπελαστεί εδώ:

<https://www.kaggle.com/code/sdi2200284/alexturingqevasion/edit/run/305187>

This leaderboard is calculated with approximately 49% of the test data. The final results will be based on the other 51%, so the final standings may be different.

#	Team	Members	Score	Entries	Last
1	sdi2300132		0.71	27	2d
2	sdi2300124		0.68	37	7h
3	AlexTuring - sdi2200284		0.66	10	5d
Your Best Entry! Your most recent submission scored 0.64, which is not an improvement of your previous score. Keep trying!					
4	sdi2300203		0.66	6	21d
5	7115112500014		0.66	14	8h

Σχήμα 7: Screenshot από το Kaggle leaderboard, όπου η υποβολή με nickname **AlexTuring** - `sdi2200284` εμφανίζεται στην 3η θέση.

Δεν θεωρώ ότι αυτό είναι κάποια αντίφαση. Σε τέτοια tasks είναι αρκετά συνηθισμένο το local best validation model να μην είναι απαραίτητα και το καλύτερο submit στο hidden leaderboard. Στη δική μου περίπτωση, περισσότερες από μία παραλλαγές της οικογένειας `qa_dual_tfidf` ήταν πολύ ανταγωνιστικές, οπότε η τελική επιλογή στηρίχθηκε και στη συνολική σταθερότητα του setup.

4.2. Συνολική ερμηνεία των αποτελεσμάτων

Αν έπρεπε να συνοψίσω την εικόνα των experiments σε μία πρόταση, θα έλεγα ότι το `qa_dual_tfidf` ήταν με διαφορά η πιο πετυχημένη κατεύθυνση. Ξεπέρασε τόσο το απλό TF-IDF baseline όσο και όλα τα Word2Vec variants. Αυτό δείχνει ότι στο συγκεκριμένο task τα sparse lexical features παραμένουν πολύ δυνατά, ειδικά όταν το μοντέλο βλέπει με ξεκάθαρο τρόπο και το question και το answer.

Πιστεύω ότι αυτό συμβαίνει για δύο λόγους. Πρώτον, τα labels δεν διαφέρουν μόνο σε επίπεδο γενικής σημασίας, αλλά και σε πιο “λειτουργικά” patterns: αν η απάντηση περιέχει συγκεκριμένο evidence, αν απαντάει στο σωστό ζητούμενο, αν ξεκινάει με deflection, αν αλλάζει θέμα κ.λπ. Δεύτερον, οι διαφορές μεταξύ *Ambivalent* και *Clear Reply* είναι συχνά πολύ τοπικές και λεπτές. Εκεί τα n-grams, τα answer openings και τα explicit alignment features φαίνεται να βοηθούν περισσότερο από ένα average Word2Vec embedding. Η γενική

αυτή εικόνα είναι και συμβατή με τη σχετική βιβλιογραφία για text representations και NLP evaluation [3].

Επίσης, τα Word2Vec-based models έδειξαν κάτι ενδιαφέρον: μερικές φορές είχαν σχετικά καλή accuracy, αλλά αισθητά χειρότερο macro-F1. Αυτό είναι ακριβώς το είδος περίπτωσης όπου η accuracy από μόνη της δίνει κάπως παραπλανητική εικόνα. Το μοντέλο μπορεί να εκμεταλλεύεται το class imbalance και να τα πηγαίνει καλά κυρίως στην *Ambivalent*, χωρίς όμως να δίνει την ίδια προσοχή στις άλλες δύο classes.

4.3. Περιορισμοί και πιθανές επεκτάσεις

Παρότι το τελικό αποτέλεσμα ήταν αρκετά καλό, υπάρχουν αρκετά σημεία που θα μπορούσαν να βελτιωθούν σε επόμενη φάση:

1. χρήση πιο ισχυρών contextual μοντέλων, π.χ. cross-encoders ή transformer setups που βλέπουν question και answer μαζί,
2. πιο στοχευμένη μοντελοποίηση της κλάσης *Ambivalent*, ίσως με ordinal ή pairwise formulation,
3. καλύτερη αξιοποίηση discourse cues, hedging, deflection markers και γενικά pragmatic information,
4. πιο έξυπνο pretraining ή καλύτερα dense embeddings, μόνο αν το extra computational cost αξίζει πραγματικά.

Παρόλα αυτά, για το scope της συγκεκριμένης εργασίας θεωρώ ότι η τελική λύση είναι αρκετά ισχυρή, πλήρως αναπαραγώγιμη και δείχνει ότι ένα καλά στημένο linear pipeline με σωστά features μπορεί ακόμα να είναι πολύ ανταγωνιστικό.

5. Συμπεράσματα

Συνολικά, η εργασία έδειξε ότι η ταξινόμηση σαφήνειας απαντήσεων σε πολιτικά QA pairs δεν είναι απλώς πρόβλημα text classification πάνω στο answer text. Χρειάζεται να ληφθεί σοβαρά υπόψη η σχέση μεταξύ question και answer. Το απλό TF-IDF baseline ήταν ήδη αξιοπρεπές, αλλά η σημαντική βελτίωση ήρθε όταν πέρασα στο `qa_dual_tfidf` και πρόσθεσα QA-aware engineered features.

Από την άλλη, τα Word2Vec-based representations ήταν χρήσιμα ως το δεύτερο ζητούμενο representation family και ως σύγκριση, αλλά δεν έφτασαν ποτέ τα καλύτερα TF-IDF runs. Αυτό δεν σημαίνει ότι είναι “κακά” γενικά· απλώς στο συγκεκριμένο task τα sparse lexical και structural cues φαίνεται να είναι πιο σημαντικά από τη συμπαγή dense σημασιολογική πληροφορία.

Τέλος, το public score **0.66** και η **top-3** θέση στο leaderboard δείχνουν ότι η προσέγγιση αυτή δεν λειτούργησε μόνο στο local validation, αλλά γενίκευσε και αρκετά καλά στο hidden evaluation. Για μένα, το πιο σημαντικό takeaway από την εργασία είναι ότι σε τέτοια NLP tasks συχνά κερδίζει όχι το πιο “βαρύ” μοντέλο, αλλά αυτό που αποτυπώνει πιο σωστά τη δομή του προβλήματος.

6. Βιβλιογραφία

Αναφορές

- [1] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*, 2013.
- [2] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [3] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd draft edition, online manuscript, 2025.
- [4] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. To appear, 2020.